Live Presentation, Demo & Defense Script

TRACK B: MACHINE LEARNING AT SCALE (HDFS → SPARK → SPARK MLLIB) • DURATION: 10-15 MINS

1. Presentation Strategy & Role Assignments

This natural speaking guide coordinates screen driving and verbal presentation. Even if one person is running the terminal, teammates speak according to their layer without getting confused.

Role	Lead Member	Assigned Script & Commands	Key Technical Defense
Terminal Driver	Shared / Any	Runs commands 1 to 7 on cues	Terminal navigation & screen sharing
Speaker 1: Storage	HDFS Lead	Commands 1 & 2 (Raw & Parquet HDFS)	Ingestion rate-limiting, 128MB block & partitioning
Speaker 2: Spark ETL	Spark Lead	Commands 3 & 4 (Spark ETL & SQL)	CPCB 16h completeness, unit conversions, diurnal peaks
Speaker 3: MLlib	ML Lead	Commands 5 & 6 (Regression & Class.)	Leak-free chronological 80/20 split, RF vs Baseline (+16.6%)

Step 1: Inspect Raw Telemetry in Hadoop HDFS — Speaker 1 (HDFS Lead)

Action: Driver types ./1_hdfs_show_raw.sh into terminal when Speaker 1 (HDFS Lead) cues.

Terminal Display (What Examiner Sees)	Speaking Script (Speaker 1 (HDFS Lead))

`$ hdfs dfs -ls -h /air_quality/raw/`
`Found 1 items`
`-rw-r--r--  1 yash supergroup  47.3 M`
`/air_quality/raw/`
`raw_hourly_measurements.jsonl`
	What to Say Naturally:
	"Good morning evaluators. We are presenting Track B: Machine Learning at Scale. As you can see, our raw sensor data is staged directly in Hadoop HDFS under /air_quality/raw/. This 47.3 Megabyte file contains 96,000 raw hourly sensor readings fetched from OpenAQ API v3 across Delhi, Bengaluru, and Hyderabad using our rate-limited ingestion client."
	
	What this Proves: *Proves real Big Data staging in HDFS with 128MB block architecture and 3x replication, rather than reading local CSVs.*

Step 2: Inspect Partitioned Columnar Lake in HDFS — Speaker 1 (HDFS Lead)

Action: Driver types ./2_hdfs_show_parquet.sh into terminal when Speaker 1 (HDFS Lead) cues.

Terminal Display (What Examiner Sees)	Speaking Script (Speaker 1 (HDFS Lead))

`$ hdfs dfs -ls -h .../hourly_aqi_parquet/`
`Found 4 items`
`_SUCCESS`
`city=Bengaluru`
`city=Delhi NCR`
`city=Hyderabad`
	What to Say Naturally:
	"Driver is now displaying our processed storage layer. Instead of slow CSVs, we persist our cleaned data as Snappy-compressed columnar Apache Parquet partitioned by city. When downstream Spark queries run on Delhi, Spark performs partition pruning, scanning only the relevant directory and cutting disk I/O significantly."
	
	What this Proves: *Demonstrates advanced data lake design, partition pruning, and columnar compression in HDFS.*

Step 3: Execute Distributed Apache Spark ETL & CPCB Math — Speaker 2 (Spark ETL Lead)

Action: Driver types ./3_spark_run_etl.sh into terminal when Speaker 2 (Spark ETL Lead) cues.

Terminal Display (What Examiner Sees)	Speaking Script (Speaker 2 (Spark ETL Lead))

`$ python3 scripts/spark_etl.py`
`================ STARTING SPARK ETL ============`
`====`
`[1/6] Ingested 96,000 raw records.`
`[3/6] Normalizing IST & gaseous units (ppb -> ug`
`/m3)`
`[5/6] Total Station-Hours: 18,521`
`Records Meeting CPCB 16h Rule: 12,792`
`Sample Output:`
`Delhi NCR | R K Puram | cpcb_aqi: 500.0 (Severe)`
`Spark ETL completed in 20.71 seconds.`
	What to Say Naturally:
	"I will now explain our Spark ETL pipeline. In 20 seconds, Spark cleans units, converts gases from ppb to ug/m3 using molecular weights, and computes rolling 24h windows. Crucially, we enforce the official Indian CPCB 16-hour completeness rule: a 24h window is only legally valid if at least 16 hours of valid data exist. Notice out of 18,521 station-hours, exactly 12,792 met this strict regulatory test."
	
	What this Proves: *Proves deep domain compliance with Indian government CPCB NAQI piecewise formulas and high-performance in-memory Spark transformations.*

Step 4: Run Distributed Spark SQL Analytics & Trend Insights — Speaker 2 (Spark ETL Lead)

Action: Driver types ./4_spark_run_analytics.sh into terminal when Speaker 2 (Spark ETL Lead) cues.
Generates: cmd4_fig1, cmd4_fig2, cmd4_fig3 in reports/figures/

Terminal Display (What Examiner Sees)	Speaking Script (Speaker 2 (Spark ETL Lead))

`$ python3 scripts/run_analytics.py`
`[1/4] City Statistics:`
`Delhi: avg PM2.5 = 121.9 | avg AQI = 382.0`
`Bengaluru: avg PM2.5 = 28.8 | avg AQI = 293.9`
`[2/4] Hazard Distribution:`
`Delhi Severe hours: 62.5%`
`Bengaluru Satisfactory: 36.6%`
`[4/4] Saved:`
`[CMD 4 - FIG 1] cmd4_fig1_diurnal_pm25_trends.`
`png`
`[CMD 4 - FIG 2] cmd4_fig2_cpcb_category_distri`
`bution.png`
`[CMD 4 - FIG 3] cmd4_fig3_city_aqi_boxplots.pn`
`g`
	What to Say Naturally:
	"Our Spark SQL queries reveal significant environmental realities. Delhi averages 121.9 ug/m3 PM2.5—over four times higher than Bengaluru. 62.5% of all Delhi hours are classified as Severe. Pointing to Figure 1 (cmd4_fig1), we see severe diurnal peaks at 08:00 AM caused by nocturnal thermal inversion trapping smoke, and an evening rush-hour peak at 08:00 PM."
	
	What this Proves: *Validates distributed analytical queries and meteorological domain understanding (thermal inversions & rush hour cycles).*

Step 5: Train Distributed Spark MLlib 24-Hour Forecasting Models — Speaker 3 (Spark MLlib Lead)

Action: Driver types ./5_spark_mllib_forecasting.sh into terminal when Speaker 3 (Spark MLlib Lead) cues.
Generates: cmd5_fig1_model_forecast_actual_vs_pred.png

Terminal Display (What Examiner Sees)	Speaking Script (Speaker 3 (Spark MLlib Lead))

`$ python3 scripts/train_forecasting_models.py`
`[2/5] Chronological Split (80% Train, 20% Test)`
`[3/5] Naive Persistence Baseline -> RMSE: 42.16`
`[5/5] Training MLlib Distributed Models:`
`  - Linear Regression: RMSE 37.86 (+10.2%)`
`  - Random Forest:  RMSE 35.17 (+16.6%)`
`  - GBT Regressor:  RMSE 37.61 (+10.8%)`
`Best Model: Random Forest (RMSE 35.17, 1.38s)`
`Saved: [CMD 5 - FIG 1] cmd5_fig1_model_forecast.`
`..`
	What to Say Naturally:
	"Now for Machine Learning at scale with Spark MLlib. Notice two critical rigor decisions: First, we used a strict 80/20 chronological block split to prevent temporal data leakage—no future records leak into the past. Second, we benchmarked against the standard Naive Persistence Baseline (RMSE 42.16). Our distributed Random Forest achieved an RMSE of 35.17—a 16.6% improvement over baseline, training in just 1.38 seconds."
	
	What this Proves: *Demonstrates leak-free time-series ML, feature engineering (lags t-1 to t-24), and beating persistence benchmarks at scale.*

Step 6: Train Distributed Spark MLlib 6-Tier Hazard Classifier — Speaker 3 (Spark MLlib Lead)

Action: Driver types ./6_spark_mllib_classification.sh into terminal when Speaker 3 (Spark MLlib Lead) cues.
Generates: cmd6_fig1_cpcb_classification_confusion_matrix.png

Terminal Display (What Examiner Sees)	Speaking Script (Speaker 3 (Spark MLlib Lead))

`$ python3 scripts/train_classification_model.py`
`Training distributed RandomForestClassifier...`
`--- Spark MLlib Classification Metrics ---`
`  Accuracy:  70.21%`
`  Weighted F1-Score:  0.676`
`  Weighted Precision: 0.674`
`  Weighted Recall:  0.702`
`Saved: [CMD 6 - FIG 1] cmd6_fig1_cpcb_classifica`
`tion...`
	What to Say Naturally:
	"Beyond numerical regression, municipal health agencies need discrete category alerts. We trained a distributed RandomForestClassifier to predict which of the 6 official CPCB hazard categories will occur 24 hours in advance. It achieves 70.21% test accuracy and a 0.676 weighted F1 score. As seen in cmd6_fig1, predictions are strongly concentrated along the diagonal, accurately identifying Severe alerts."
	
	What this Proves: *Demonstrates multi-class classification at scale and practical public health decision support.*

Step 7: Master Automated Pipeline & Wrap-Up

Action if examiner asks for an end-to-end run: Driver executes ./7_run_entire_pipeline.sh.
Speaker 1 Conclusion: *"In conclusion, our project demonstrates a complete, fault-tolerant distributed system under Track B: ingestion from real-world telemetry, raw storage in HDFS, strict regulatory CPCB mathematical transformation in Spark ETL, and distributed predictive forecasting with Spark MLlib. We are now happy to take your questions."*

2. Evaluator Viva Q&A; Defense Cheat Sheet

Be prepared with these concise, mathematically and architecturally rigorous answers:

Question	Examiner Intent	Your Bulletproof Answer

Why Parquet over CSV in HDFS?	Tests distributed storage knowledge	Parquet is columnar and Snappy-compressed, supporting **Partition Pruning** and dictionary encoding. Queries filtering by city skip irrelevant directories entirely, minimizing disk I/O.
Why Chronological split over K-Fold?	Tests time-series ML integrity	Air quality is auto-correlated time series. Standard K-fold or random shuffling leaks future weather and pollution data into the past, causing severe data leakage and fake high scores.
What is CPCB 16-Hour rule?	Tests domain compliance	CPCB guidelines require at least 16 hourly sensor readings in a 24-hour moving window for an exposure sub-index to be legally valid. Missing data below 16 hours is marked invalid.
Why Random Forest beat GBT?	Tests ML architecture comparison	Air quality telemetry has high sensor noise and sudden peaks (inversion, dust, traffic). GBT optimizes residuals and overfits outliers, while Random Forest's bagging reduces variance and handles noise better.
How does this scale to 500+ cities?	Tests Big Data scalability	Because every single transformation is written in distributed PySpark over HDFS, scaling requires zero code changes—only adding worker nodes and adjusting executor memory in spark-submit.